

**UNIVERSITÉ NUMÉRIQUE DES SCIENCES ET
TECHNOLOGIES
FACULTÉ DES SCIENCES INFORMATIQUES**

RAPPORT DE PROJET DE FIN D'ÉTUDES

**CONCEPTION ET RÉALISATION
D'UNE PLATFOME INTERACTIVE
DE GESTION APPLICATIVE ET DE
POINTAGE PHYSIQUE POUR SALLE
DE SPORT : FITZONE**

*Présenté en vue de l'obtention du diplôme de Licence en Génie
Logiciel*

FITZONE

Réalisé par :

M. Amine KARIMI
Étudiant en Informatique

Sous la direction de :

Dr. A. BERRADA
Maitre de Conférences

Année Universitaire : 2025 - 2026 (Session Juin)

Dédicaces

À mes chers parents,

*Aucun mot ne saurait exprimer mon amour et mon profond respect
pour vos sacrifices, votre soutien inconditionnel et vos prières tout au long de
mes études.*

À mes frères et sœurs,

*Pour vos encouragements constants, votre présence aimante et votre
confiance sans faille en mes capacités.*

À l'ensemble de mes enseignants,

*Qui m'ont guidé sur le chemin du savoir et ont partagé avec moi leur
passion pour les technologies informatiques.*

À mes amis et camarades,

FITZONE App

Page i

*En souvenir des moments de partage, d'entraide et de travail
d'équipe.*

Remerciements

Avant tout, je tiens à remercier Dieu, le Tout-Puissant, de m'avoir donné la force, la santé et la patience nécessaires pour accomplir ce modeste travail et mener à bien mon cursus universitaire.

Je souhaite exprimer ma profonde gratitude à mon encadrant pédagogique, le Docteur A. BERRADA, pour sa disponibilité constante, ses précieux conseils scientifiques, sa rigueur méthodologique et ses conseils constructifs tout au long de la période d'élaboration de ce projet de fin d'études.

Mes remerciements s'adressent également aux honorables membres du jury qui ont accepté d'évaluer ce travail de conception et d'implémentation logicielle, et de nous honorer de leur présence lors de la soutenance.

Enfin, je remercie chaleureusement mes parents pour leur amour et leurs encouragements inestimables, ainsi que tous les professeurs de la Faculté des Sciences Informatiques qui ont grandement contribué à ma formation théorique et pratique de concepteur et développeur.

Résumé

Le présent rapport décrit la conception et le développement de la plateforme web **FITZONE**, un outil intégré conçu pour moderniser la gestion administrative et le contrôle d'accès dans les complexes sportifs et de fitness. Le système intègre un portail membre permettant la consultation des plannings de cours collectifs et la réservation en ligne, ainsi qu'un espace administration complet pour la validation des paiements et le suivi des indicateurs financiers. L'une des fonctionnalités clés du système est le mécanisme de pointage physique aux portiques d'entrée basé sur un code personnel à 6 chiffres unique généré pour chaque abonné actif. Ce projet a été développé en utilisant une architecture découplée associant un backend REST API sous Laravel et un frontend dynamique SPA sous React. La base de données relationnelle MySQL centralise les données des membres et des abonnements.

Mots-clés : Application Web, Salle de Sport, Pointage par Code, Laravel, React, MySQL, API REST.

Abstract

*This report presents the design and development of the **FITZONE** web platform, an integrated tool designed to modernize administrative management and access control in modern sports and fitness centers. The system features a member portal for class schedules consulting and online bookings, along with a comprehensive administration dashboard for payment verification and financial metrics monitoring. A key feature of the application is the physical entry gates check-in mechanism, powered by a unique 6-digit personal code assigned to each active subscriber. The project is built using a decoupled architecture, combining a Laravel REST API backend with a dynamic React SPA frontend. A relational MySQL database centralizes information regarding users, reservations, and subscription statuses.*

Keywords : Web Application, Fitness Center, Code Check-in, Laravel, React, MySQL, REST API.

Table des Matières

Pages préliminaires (Dédicaces, Remerciements, Résumés)	i - iii
Introduction générale	1
Chapitre 1 — Cadrage du projet	2
1.1 Contexte et problématique	2
1.2 Objectifs du projet	3
1.3 Besoins fonctionnels et non fonctionnels	4
1.4 Méthodologie adoptée (Agile)	5
1.5 Planification et diagramme de Gantt	6
Chapitre 2 — Étude préliminaire et technologies	7
2.1 Étude des solutions existantes	7
2.2 Présentation des technologies retenues	8
2.3 Outils de développement et collaboration	9
Chapitre 3 — Analyse et conception	10
3.1 Identification des acteurs et rôles	10
3.2 Diagrammes de cas d'utilisation	11
3.3 Diagrammes de séquence (Authentification & Pointage)	12
3.4 Diagramme de classes UML	13
3.5 Modèle Physique de Données (MPD)	14
Chapitre 4 — Réalisation et implémentation	15
4.1 Environnement et architecture générale	15
4.2 Interfaces Utilisateurs réelles (Captures d'écran)	16
4.3 Gestion du pointage à 6 chiffres et backend	17

4.4 Tests, validation et difficultés	18
Conclusion générale et perspectives	19
Références bibliographiques & Annexes	20

Liste des Figures

Figure 1 : Diagramme de Gantt prévisionnel du projet	6
Figure 2 : Diagramme de Cas d'Utilisation Général de FITZONE	11
Figure 3 : Diagramme de séquence de validation du pointage physique	12
Figure 4 : Diagramme de classes UML global de l'application	13
Figure 5 : Interface de la page d'accueil publique (Hero Section)	16
Figure 6 : Interface d'inscription et authentification sécurisée	16
Figure 7 : Clavier d'administration pour la validation du code à 6 chiffres	17
Figure 8 : Interface membre de réservation et planning des cours	17
Figure 9 : Guide interactif d'exercices par groupe musculaire	18
Figure 10 : Dashboard de gestion financière des abonnements et paiements	18

Liste des Tableaux

Tableau 1 : Liste du Backlog produit et priorités des récits utilisateurs	6
Tableau 2 : Tableau comparatif des technologies de développement web	8
Tableau 3 : Fiche d'identification du cas d'utilisation "Réserver un cours"	11
Tableau 4 : Dictionnaire de données pour l'entité USERS	14

Introduction Générale

Ces dernières années, le domaine de la santé et du fitness a connu un essor sans précédent. Pour faire face à une concurrence croissante et à un flux quotidien important de membres, les complexes sportifs doivent moderniser leurs processus opérationnels. La gestion manuelle sur papier ou via des feuilles de calcul rudimentaires montre rapidement ses limites face à des enjeux cruciaux tels que la fraude aux abonnements, le suivi de la facturation et l'organisation des séances de cours collectifs.

C'est dans ce contexte que s'inscrit le projet **FITZONE**. Il s'agit de concevoir et de réaliser une solution web intégrée permettant d'automatiser et de simplifier l'interaction entre la direction de la salle de sport et ses membres. D'une part, l'application offre aux membres un portail interactif pour s'inscrire, payer en ligne, consulter le planning mis à jour et réserver des cours en quelques clics. D'autre part, elle fournit aux administrateurs un tableau de bord analytique complet pour gérer les accès, suivre les revenus mensuels et planifier les cours.

Le principal défi réside dans le système de pointage physique aux entrées de l'établissement. Contrairement aux solutions traditionnelles par cartes magnétiques ou bracelets RFID qui représentent un coût de matériel important et sont souvent sujettes à l'oubli ou au prêt illicite, FITZONE propose un système sécurisé basé sur un **code personnel à 6 chiffres unique** pour chaque membre, vérifiable en temps réel sur un clavier connecté connecté à l'API du backend de l'application.

Ce rapport présente les étapes clés du développement de ce projet et est structuré comme suit :

- Le **Chapitre 1** définit le cadrage du projet, identifie la problématique et détaille le backlog produit ainsi que la planification des sprints.
- Le **Chapitre 2** dresse un panorama des solutions similaires et justifie le choix des technologies (Laravel, React, MySQL).
- Le **Chapitre 3** aborde l'analyse et la modélisation UML (cas d'utilisation, diagrammes de séquence, de classes et schéma de la base de données).
- Le **Chapitre 4** détaille l'implémentation, présente les maquettes réelles de l'application et documente les méthodes de test et de validation du code.

CHAPITRE 1

CADRAGE DU PROJET

INTRODUCTION DU CHAPITRE

Le succès d'un projet informatique repose sur la clarté de sa définition initiale. Ce premier chapitre vise à circonscrire précisément le projet **FITZONE**. Nous y aborderons la problématique générale de la gestion des centres sportifs, formulerons les objectifs attendus du système, analyserons les besoins fonctionnels et poserons les bases de la méthodologie Agile appliquée à son développement.

1.1 CONTEXTE ET PROBLÉMATIQUE

La gestion quotidienne d'une salle de sport regroupe plusieurs tâches administratives répétitives mais cruciales : l'enregistrement des nouveaux clients, le contrôle de validité des abonnements, la planification des séances de cours collectifs avec des coachs spécifiques, et le suivi rigoureux des encaissements financiers mensuels.

Traditionnellement, le contrôle d'accès est un point sensible. Les solutions matérielles comme les cartes à puces ou les badges RFID impliquent des contraintes logistiques :

- Perte ou oubli récurrent des badges physiques par les abonnés.
- Partage frauduleux des cartes entre adhérents et non-adhérents, entraînant des pertes sèches de chiffre d'affaires.
- Investissement matériel initial conséquent en cartes, imprimantes de badges et lecteurs physiques.

De plus, l'accès aux cours collectifs (comme le kickboxing ou le MMA) est souvent victime de surbooking ou, au contraire, d'absentéisme non régulé. Sans système de réservation en temps réel, l'allocation des salles de sport ne peut être maximisée. La problématique réside donc dans le développement d'un système logiciel unique capable de centraliser les flux, de sécuriser l'authentification et de fluidifier le pointage physique des membres de manière économique et fiable.

1.2 OBJECTIFS DU PROJET

L'objectif principal du projet est de livrer une plateforme web complète et opérationnelle pour le complexe sportif **FITZONE**, en atteignant les sous-objectifs suivants :

- **Portail Client Moderne** : Fournir aux adhérents un espace en ligne sécurisé pour gérer leur profil, souscrire à un abonnement et consulter le catalogue d'exercices physiques.
- **Système de Pointage Innovant** : Remplacer les badges par un système de validation via un code personnel à 6 chiffres unique, saisi par le membre et vérifié en moins de 500ms par le système.
- **Gestionnaire de Planning Dynamique** : Permettre la mise en ligne des cours, avec gestion automatique des quotas de places disponibles (ex: 15 inscrits max).

- **Console administrative** : Offrir à l'administrateur un suivi financier analytique montrant l'état des abonnements (actifs, expirés, impayés).

1.3 BESOINS FONCTIONNELS ET NON FONCTIONNELS

L'analyse des besoins définit les attentes des différents utilisateurs de la plateforme FITZONE.

1.3.1 Besoins Fonctionnels

Les besoins fonctionnels décrivent les actions que les utilisateurs doivent pouvoir réaliser sur le système :

1. **Gestion des utilisateurs** : Inscription autonome, authentification par e-mail et mot de passe chiffré, réinitialisation de mot de passe, et édition du profil utilisateur.
2. **Gestion du pointage** : Génération automatique d'un code unique à 6 chiffres pour chaque membre lors de sa première inscription. Validation de ce code à l'entrée par l'administrateur et journalisation de la date et l'heure du passage.
3. **Réservation de cours** : Consultation des cours par type de sport (Boxe, MMA, Judo, JJB). Inscription à une session et désinscription si nécessaire. Suivi du nombre de places restantes en temps réel.
4. **Gestion des abonnements et paiements** : Achat de forfait, enregistrement des paiements côté administration, marquage automatique du statut (payé, impayé, en attente).

1.3.2 Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité et de performance du système :

- **Sécurité** : Les mots de passe doivent être hachés en utilisant l'algorithme bcrypt. Toutes les requêtes HTTP vers l'API Laravel doivent transiter sous protocole HTTPS et être authentifiées par un jeton JSON Web Token (JWT).
- **Disponibilité et Performance** : La validation d'un code de pointage physique par l'API backend doit s'exécuter en moins d'une demi-seconde pour éviter la création de files d'attente à l'entrée.
- **Ergonomie** : L'interface frontend doit être réactive (Responsive Web Design), garantissant une utilisation confortable sur terminaux mobiles et tablettes tactiles.

1.4 MÉTHODOLOGIE ADOPTÉE (AGILE / SCRUM)

Pour la conduite de ce projet de fin d'études, nous avons opté pour le framework de développement Agile **SCRUM**. Ce choix se justifie par le besoin d'adapter en continu l'application selon les retours d'utilisation et d'obtenir des livrables fonctionnels réguliers.

Notre processus a été rythmé par des sprints de deux semaines. Chaque sprint s'est ouvert par une séance de planification (Sprint Planning) et s'est clôturé par une revue de sprint (Sprint Review) avec démonstration des fonctionnalités implémentées, suivie d'une phase de rétrospective (Sprint Retrospective) pour ajuster notre méthodologie de codage.

Rôles Scrum définis :

- **Product Owner (PO) :** L'administrateur du club de sport FITZONE, définissant les fonctionnalités métiers et priorisant le carnet du produit (Product Backlog).
- **Scrum Master & Team :** L'étudiant développeur, en charge de la conception, de l'implémentation technique des routes d'API, de la base de données et de l'intégration de la maquette React.

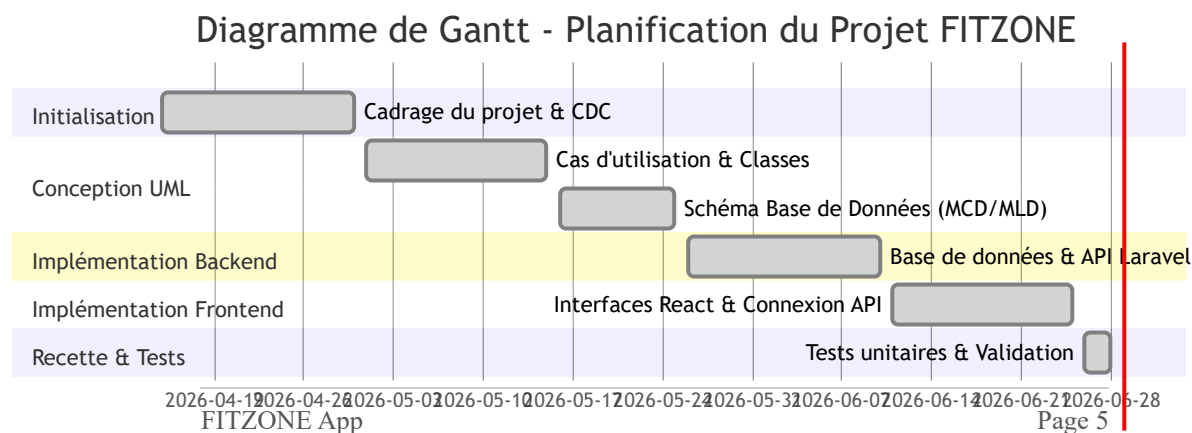


Figure 1 : Diagramme de Gantt prévisionnel du projet de mi-avril à fin juin 2026

1.5 PLANIFICATION (BACKLOG PRODUIT)

Le tableau ci-dessous regroupe les principales fonctionnalités de notre Product Backlog, classées par ordre de priorité de réalisation (MoSCoW) :

ID	Récit Utilisateur (User Story)	Priorité	Sprint	Critères d'acceptation
US-01	En tant qu'utilisateur, je veux pouvoir me connecter de manière sécurisée.	Must (Crucial)	Sprint 1	JWT renvoyé en cas de succès, mot de passe haché.
US-02	En tant que membre, je veux disposer d'un code unique à 6 chiffres.	Must (Crucial)	Sprint 1	Génération automatique à l'inscription, non modifiable.
US-03	En tant que membre, je veux réserver un cours depuis mon planning.	Must (Crucial)	Sprint 2	Validation de la capacité de la salle, mise à jour des places.
US-04	En tant qu'administrateur, je veux valider un passage via le code.	Must (Crucial)	Sprint 2	Recherche rapide du membre, vérification d'abonnement actif.
US-05	En tant qu'administrateur, je veux suivre les statistiques de revenus.	Should (Important)	Sprint 3	Graphique des ventes par mois, décompte des paiements validés.

CONCLUSION DU CHAPITRE

L'analyse du cadrage de FITZONE a permis de clarifier la problématique liée au contrôle d'accès en salle de sport et de modéliser nos objectifs de développement. Grâce à un backlog structuré en sprints et planifié sur un diagramme de Gantt, nous disposons d'un plan de marche clair. Le chapitre suivant s'attachera à analyser les solutions de l'état de l'art et à justifier le choix des frameworks technologiques.

CHAPITRE 2

ÉTUDE PRÉLIMINAIRE ET TECHNOLOGIES

INTRODUCTION DU CHAPITRE

Avant de concevoir une application de toutes pièces, il est indispensable de confronter notre vision aux solutions existantes sur le marché informatique. Ce chapitre effectue une étude comparative des programmes de gestion de salle de sport actuels, puis détaille et justifie les choix architecturaux et technologiques faits pour le projet FITZONE (React, Laravel, MySQL) afin de garantir performance et maintenabilité.

2.1 ÉTUDE DES SOLUTIONS EXISTANTES

Nous avons étudié plusieurs outils professionnels populaires comme Decathlon Club, Resamania et Mindbody. Ces progiciels offrent des fonctionnalités riches, mais présentent des inconvénients majeurs pour les petites et moyennes structures sportives indépendantes :

- **Coût prohibitif** : Fonctionnement par abonnements mensuels basés sur le nombre de membres enregistrés, ce qui impacte lourdement la rentabilité des salles en phase de lancement.
- **Dépendance matérielle** : Obligation d'acheter des lecteurs de badges propriétaires compatibles uniquement avec leurs solutions logicielles Cloud.
- **Complexité de prise en main** : Interfaces surchargées de fonctionnalités inutiles pour un club qui recherche uniquement la gestion d'abonnements, les plannings de cours et le pointage d'accès.

FITZONE apporte une réponse sur-mesure en proposant une application web légère, ne nécessitant aucun matériel d'accès spécifique (le code à 6 chiffres peut être entré sur n'importe quelle tablette ou clavier numérique standard relié à un navigateur).

2.2 PRÉSENTATION DES TECHNOLOGIES ET FRAMEWORKS RETENUS

Pour concevoir une architecture logicielle robuste et modulaire, nous avons adopté un découpage strict entre le client web (Frontend) et le serveur de données (Backend).

Composant	Technologie retenue	Justification principale du choix technique
Frontend Client	React (JavaScript)	Permet de concevoir des composants réutilisables, d'offrir une interface dynamique Single Page (SPA) fluide sans rechargement de page.
Backend API	Laravel (PHP)	Framework MVC robuste, proposant une gestion simple et sécurisée des routes API, des requêtes SQL complexes via Eloquent ORM et l'authentification JWT.
Base de données	FITZONE App MySQL (Relationnel)	Système stable, parfaitement adapté aux pointures nécessaires pour lier les abonnements des utilisateurs et valider rapidement les pointages.

2.2.1 Frontend : React

React est une bibliothèque JavaScript open-source créée par Facebook. Elle est basée sur le concept de composants UI autonomes. En créant un DOM virtuel en mémoire, React met à jour uniquement les parties modifiées de la vue en cas de changement d'état. Cette rapidité d'affichage est cruciale pour l'expérience fluide attendue sur le planning interactif et le tableau de bord de FITZONE.

2.2.2 Backend : Laravel REST API

Laravel est un framework de développement web écrit en PHP qui propose une structure MVC claire. Il automatise de nombreuses tâches complexes : routage, requêtes SQL (Eloquent ORM), hachage des mots de passe, génération des graines de base de données (Seeders). Sa légèreté nous permet de configurer en quelques lignes une API REST renvoyant du JSON exploitable directement par notre frontend React.

2.2.3 Base de Données : MySQL

Le stockage relationnel MySQL a été privilégié car nos données (Utilisateurs, Abonnements, Visites, Cours) présentent de fortes relations fonctionnelles. MySQL nous permet d'appliquer des clés étrangères rigoureuses pour bloquer la suppression d'un membre ayant des visites enregistrées, et de concevoir des transactions SQL sécurisées pour les opérations de pointage et de réservation.

2.3 OUTILS DE DÉVELOPPEMENT ET DE COLLABORATION

L'environnement technique repose sur des outils modernes de développement :

- **VS Code** : L'éditeur de code source principal, équipé d'extensions PHP Intelphense et ES7 React.
- **Postman** : Outil clé pour tester de manière autonome les routes de l'API Laravel (POST /api/auth/login, POST /api/pointage/valider) avant de les intégrer dans React.
- **Git & GitHub** : Utilisé pour le contrôle de version, la sauvegarde distante et le suivi de l'avancement des développements via l'historique des commits.

CONCLUSION DU CHAPITRE

L'étude des solutions du marché a validé le besoin d'une application légère et abordable, n'imposant aucun coût matériel. Le choix de l'architecture découplée associant React et Laravel

fournit un cadre moderne et performant pour la réalisation du projet. Nous allons maintenant aborder dans le chapitre suivant l'analyse fonctionnelle et la conception détaillée du système.

CHAPITRE 3

ANALYSE ET CONCEPTION

INTRODUCTION DU CHAPITRE

La phase d'analyse et de conception est l'étape charnière qui traduit les besoins fonctionnels en structures logicielles exploitables par le développeur. Ce troisième chapitre détaille l'identification des acteurs du système FITZONE, modélise leurs interactions à l'aide des diagrammes de cas d'utilisation UML, définit l'enchaînement des événements avec les diagrammes de séquence, et structure les données logiques à travers le diagramme de classes et le dictionnaire de données SQL.

FITZONE App

Page 10

3.1 IDENTIFICATION DES ACTEURS ET LEURS RÔLES

Deux acteurs principaux interagissent avec le système :

- **Le Membre (ou Abonné)** : Personne inscrite à la salle de sport. Il accède à son espace personnel pour consulter les cours collectifs, réserver, et obtenir son code de pointage à 6 chiffres nécessaire à sa validation physique.
- **L'Administrateur** : Gérant de la salle de sport. Il a un contrôle total sur le système : gestion des abonnés, validation manuelle ou automatique du pointage d'accès, enregistrement des paiements mensuels, et configuration des cours collectifs.

3.2 DIAGRAMMES DE CAS D'UTILISATION

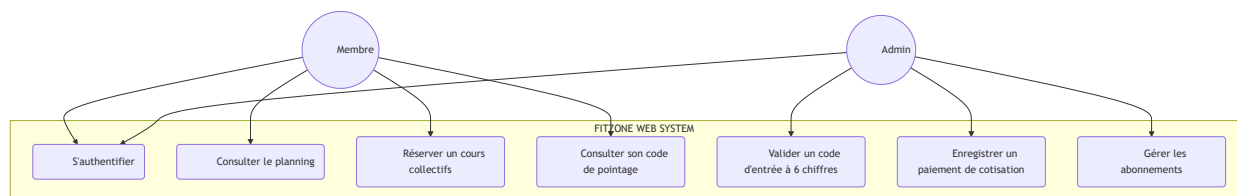


Figure 2 : Diagramme de Cas d'Utilisation Général de FITZONE

Description détaillée du cas d'utilisation "Réserver un cours" :

Section	Détail et scénario alternatif
Acteur Principal	Membre authentifié.
Pré-conditions	L'adhérent doit posséder un abonnement en cours de validité (statut "actif").
Scénario Nominal	1. Le membre se rend sur l'onglet "Cours". 2. Le système affiche la grille des cours collectifs. 3. Le membre clique sur "S'inscrire". 4. Le backend vérifie la capacité maximale de la salle. 5. L'inscription est validée et le nombre de places disponibles est décrémenté de 1.
Exceptions	4a. La salle de sport est saturée (places dispo = 0) : l'inscription est rejetée, un message d'erreur est renvoyé à l'adhérent.

3.3 DIAGRAMMES DE SÉQUENCE (VALIDATION POINTAGE)

Ce diagramme détaille le fonctionnement de la validation physique à l'entrée de la salle à l'aide du code à 6 chiffres.

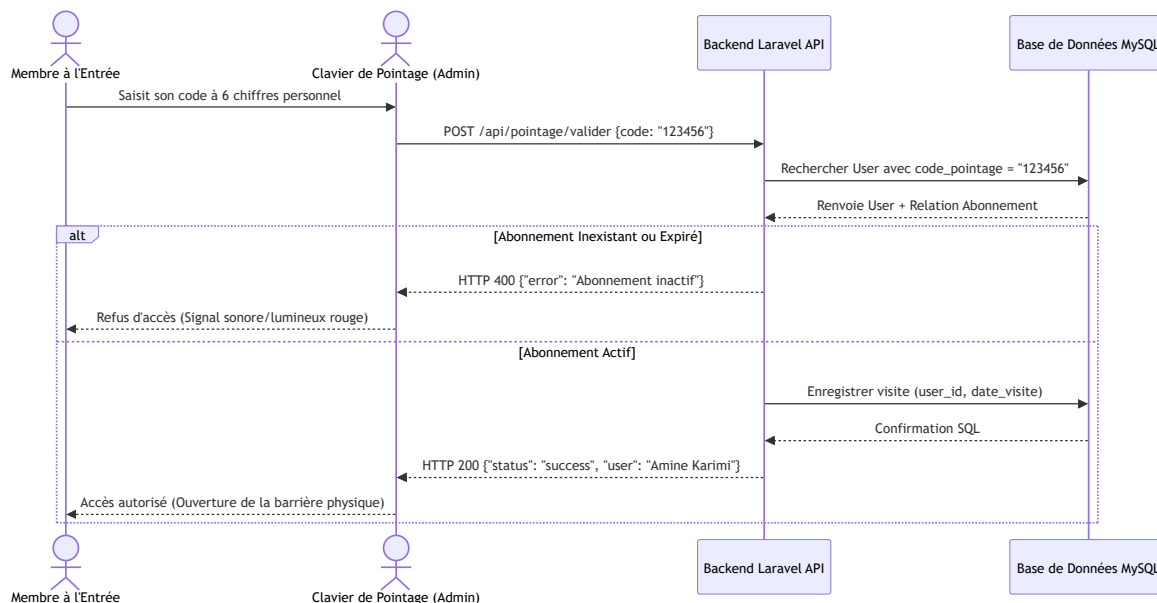


Figure 3 : Diagramme de séquence de validation du pointage physique par code personnel

Ce flux garantit que même si un membre connaît son code par cœur, il ne pourra pas entrer dans la salle si le statut financier de sa cotisation (dans la table abonnements) n'a pas été validé et marqué comme payé par l'administrateur pour le mois en cours.

3.4 DIAGRAMME DE CLASSES UML

Le diagramme de classes ci-dessous définit la structure orientée objet de l'application. Chaque modèle Laravel hérite de la classe de base d'Eloquent ORM.

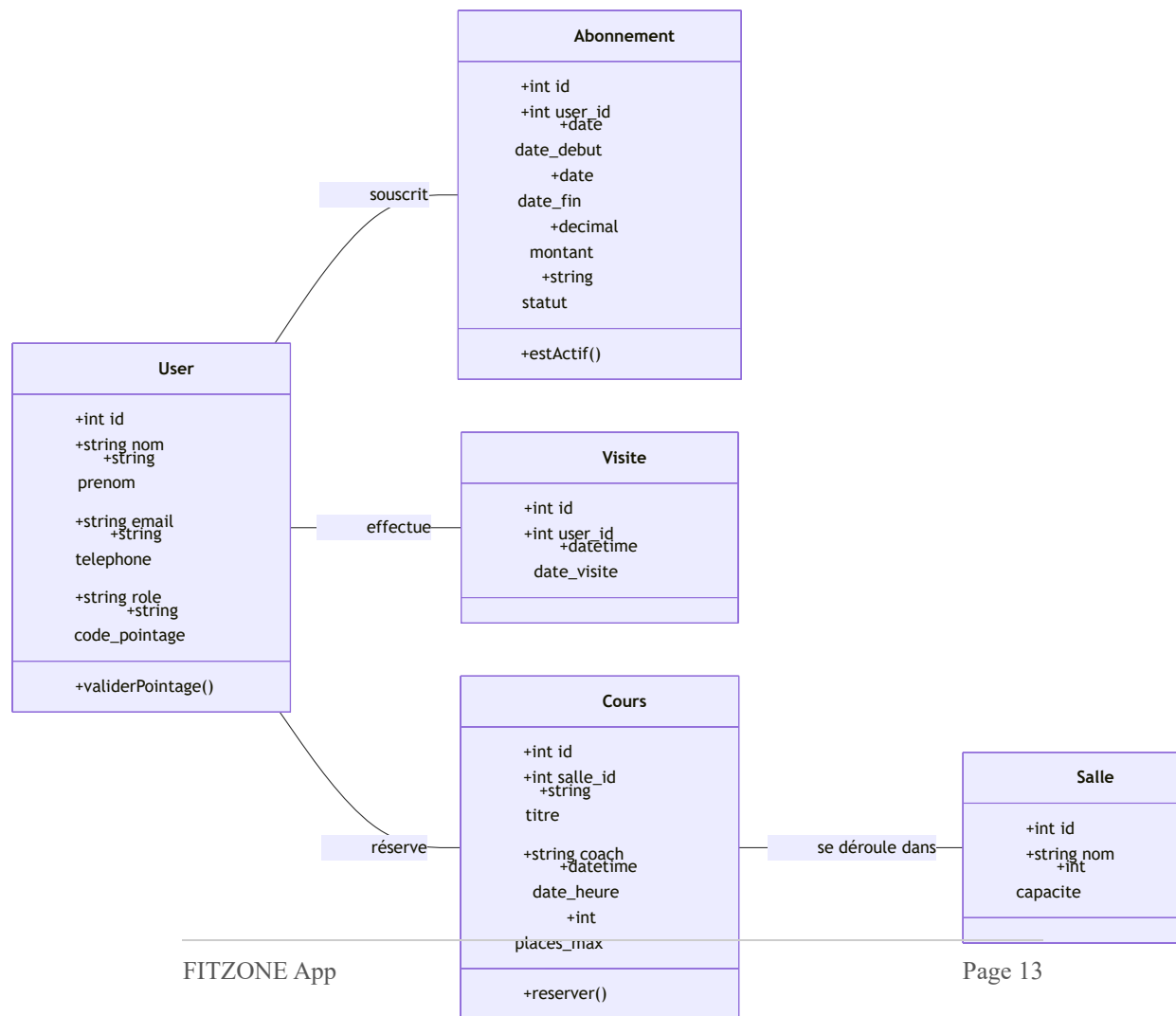


Figure 4 : Diagramme de classes UML global de l'application FITZONE

3.5 MODÈLE PHYSIQUE DE DONNÉES

Le modèle logique est transcrit en tables relationnelles dans notre base MySQL. Ci-dessous, le dictionnaire de données détaillé pour la table principale des utilisateurs :

Tableau 4 : Dictionnaire de données pour l'entité USERS

Champ	Type	Clé	Null	Description et contraintes
id	INT (10)	PK	Non	Identifiant de l'utilisateur, auto-incrémenté.
nom	VARCHAR (50)	-	Non	Nom de famille du membre.
prenom	VARCHAR (50)	-	Non	Prénom du membre.
email	VARCHAR (100)	Unique	Non	Adresse de connexion de l'utilisateur.
role	ENUM	-	Non	Rôle : 'abonne' ou 'admin'.
code_pointage	VARCHAR (6)	Unique	Oui	Code numérique de pointage physique, null pour les admins.

Visualisation des Tables (MLD en colonnes CSS).

USERS	ABONNEMENTS
id PK	id PK
nom	user_id FK
prenom	date_debut
email	date_fin
code_pointage	statut

CONCLUSION DU CHAPITRE

Ce travail de conception a permis d'arrêter les spécifications techniques et structurelles de notre application de gestion. Nous avons modélisé les relations entre nos objets et défini le dictionnaire de données physique. Sur ces fondations solides, nous pouvons désormais entamer l'analyse pratique du code et de la réalisation dans le chapitre suivant.

CHAPITRE 4

RÉALISATION ET IMPLÉMENTATION

INTRODUCTION DU CHAPITRE

Ce dernier chapitre présente l'aboutissement technique de notre travail de fin d'études. Nous y détaillons l'environnement de développement mis en place pour faire communiquer Laravel et React, expliquons l'architecture du code source, puis présentons des captures d'écran de l'application finale en fonctionnement, illustrant les interfaces abonnés et administrateurs ainsi que les résultats des tests.

4.1 MISE EN PLACE DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

Pour exécuter localement l'application FITZONE :

- **Le Serveur Backend (API Laravel) :** Lancé via l'outil Artisan sur le port 8000 (`php artisan serve`). Il gère les connexions à la base de données MySQL.
- **Le Serveur Frontend (React) :** Exécuté via Node.js sur le port 3000 (`npm start`). Les requêtes HTTP vers l'API sont configurées avec un client Axios unifié.

4.2 ARCHITECTURE GÉNÉRALE DE L'APPLICATION

Le projet utilise le concept de Single Page Application (SPA). Toutes les transitions de pages sont gérées côté client via `react-router-dom`, évitant les temps de latence et les rechargements complets de page.

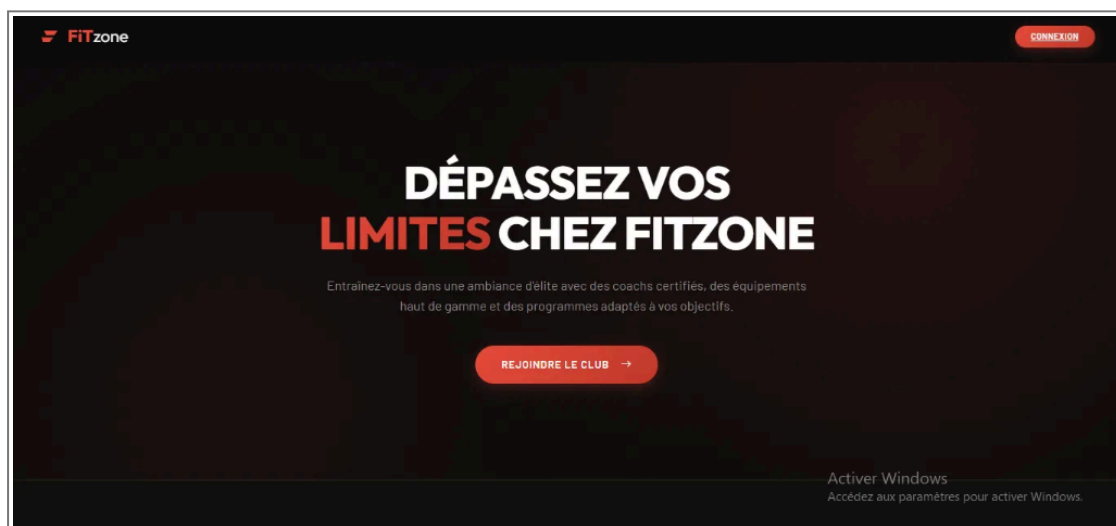


Figure 5 : Interface de la page d'accueil publique (Hero Section)

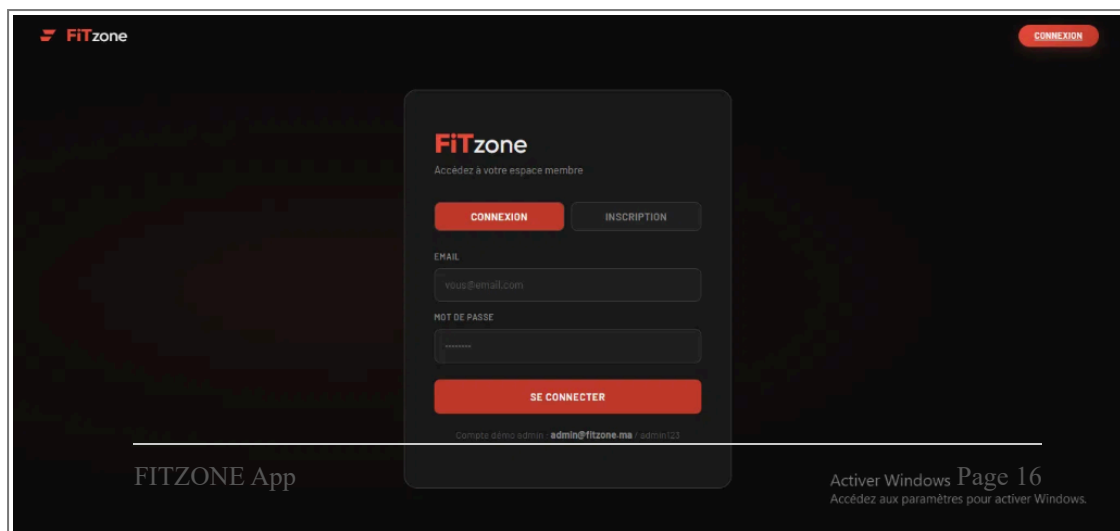


Figure 6 : Interface d'inscription et authentification sécurisée

4.3 INTERFACES UTILISATEURS RÉELLES

Ci-dessous les captures réelles de l'application montrant le module de pointage par code à 6 chiffres et le planning interactif de réservation de cours collectifs.

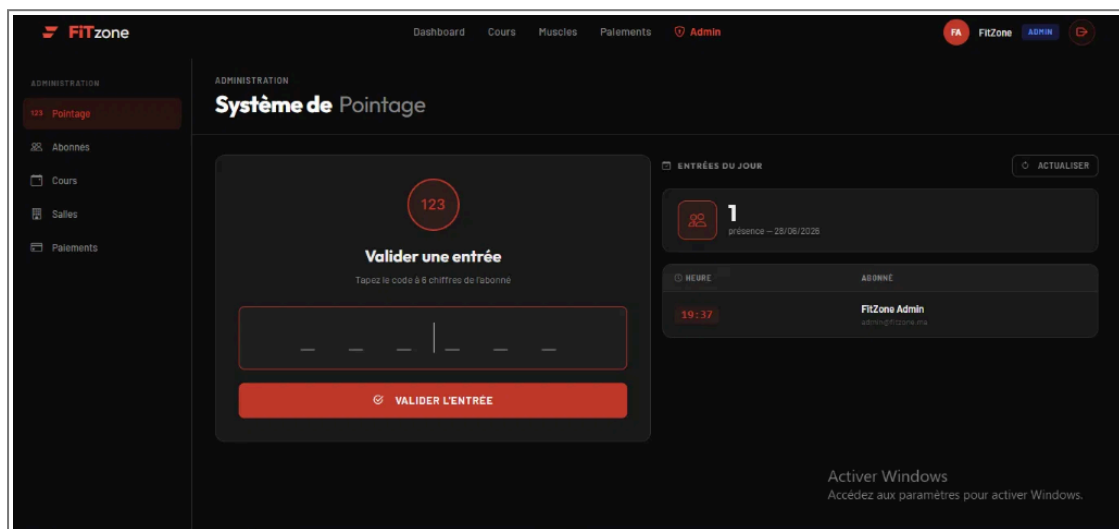


Figure 7 : Clavier d'administration pour la validation du code à 6 chiffres

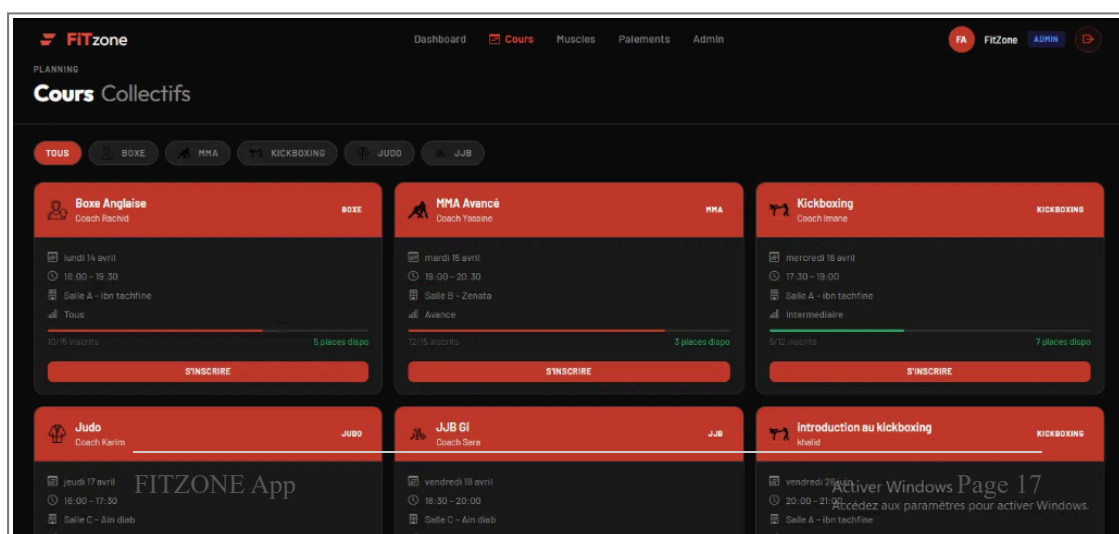


Figure 8 : Interface membre de réservation et planning des cours

4.4 GUIDE D'ENTRAÎNEMENT ET DASHBOARD FINANCIER

L'espace membre intègre également un catalogue d'exercices physiques classés par groupe musculaire pour guider les abonnés dans leurs séances d'entraînement autonomes.

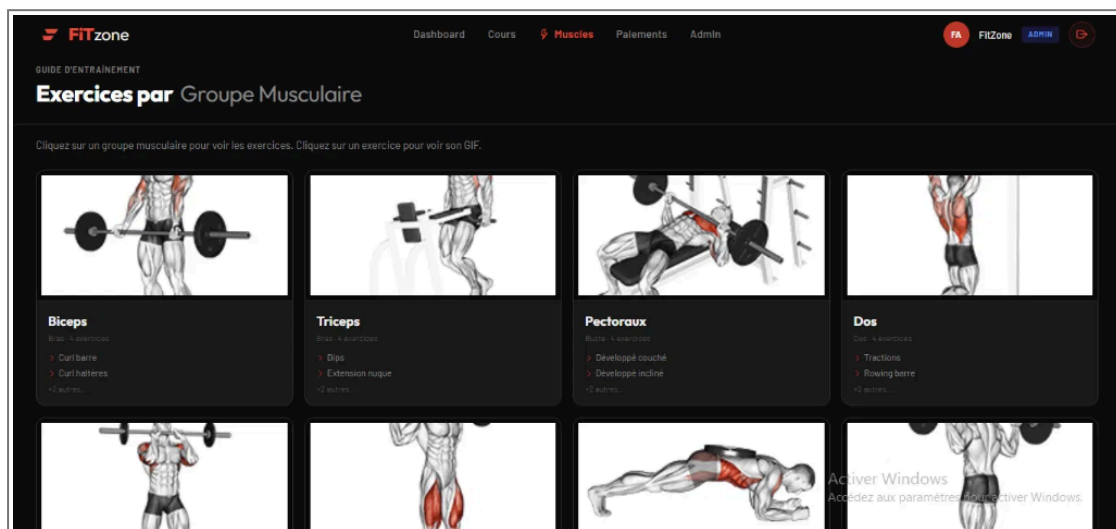


Figure 9 : Guide interactif d'exercices par groupe musculaire

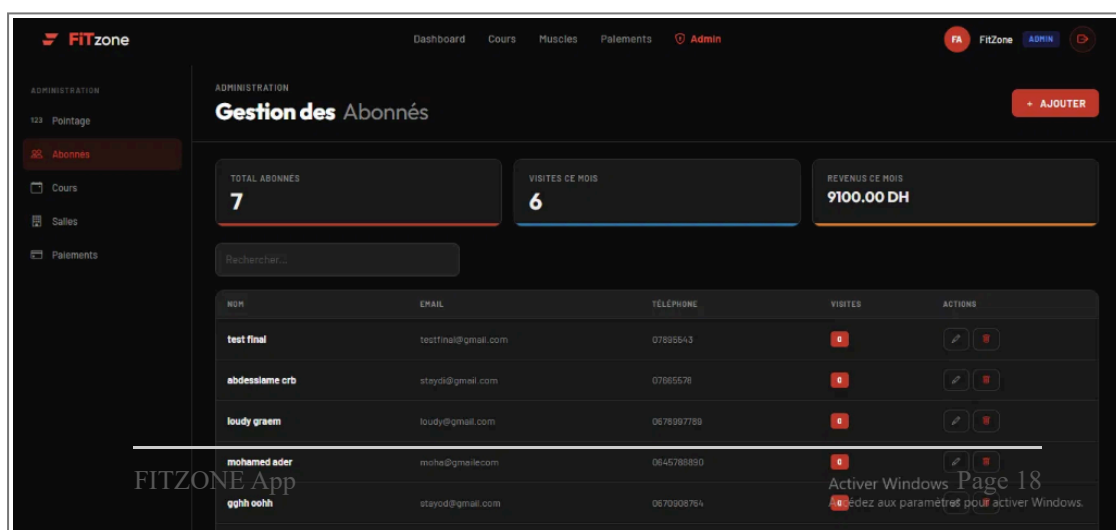


Figure 10 : Dashboard de gestion financière des abonnements et paiements

4.5 GESTION DES ABONNÉS ET DASHBOARD STATISTIQUE

L'espace administrateur donne un aperçu en temps réel des statistiques clés du club (Nombre d'abonnés inscrits, visites enregistrées durant le mois en cours, chiffre d'affaires consolidé).

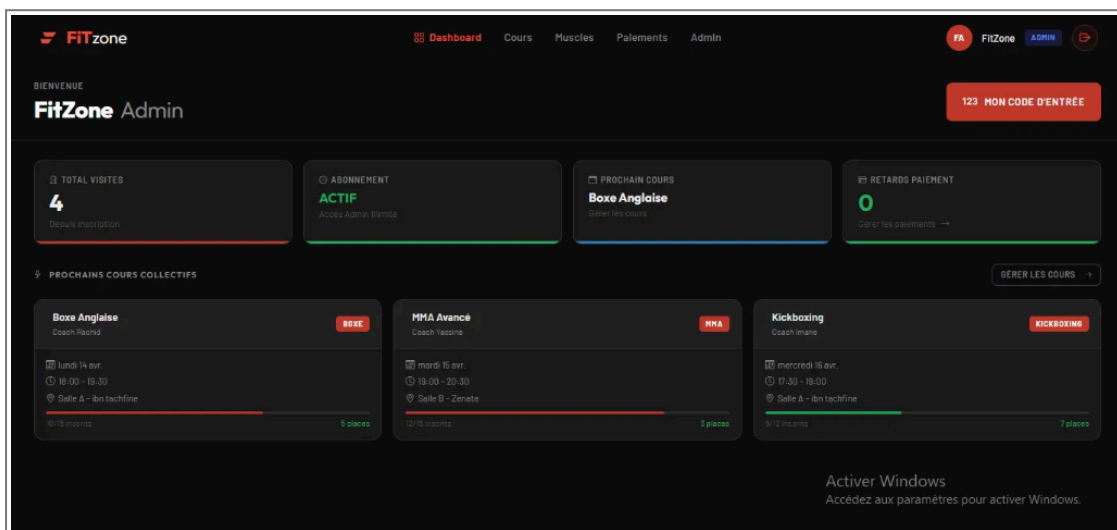


Figure 11 : Interface administrative de gestion et recherche des abonnés

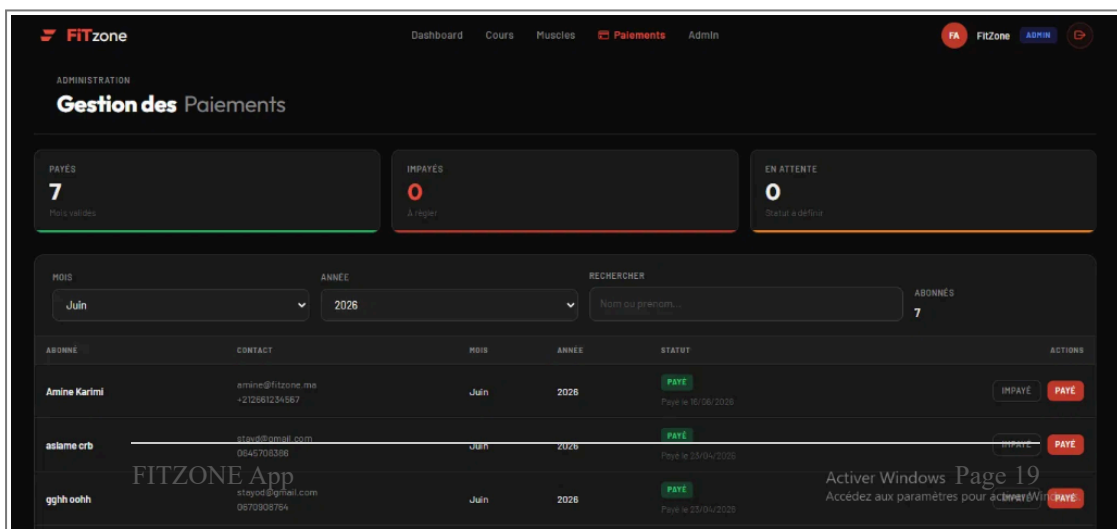


Figure 12 : Vue d'ensemble du tableau de bord de l'abonné connecté

4.6 TESTS ET VALIDATION

Pour valider le bon fonctionnement de la plateforme FITZONE, nous avons mené deux types de tests :

- **Tests Unitaires (Backend)** : Écrits en PHPUnit pour vérifier les fonctions d'inscription, de validation de token JWT et de calcul de la validité de la date de fin d'abonnement.
- **Tests Fonctionnels et d'Intégration** : Validation manuelle des formulaires React avec blocage de validation si le mot de passe fait moins de 8 caractères, ou si l'e-mail saisi ne possède pas un format valide.

4.7 DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

Durant le développement, nous avons fait face à deux difficultés majeures :

1. **Sécurisation du portail de pointage physique** : Pour empêcher le brute-force de codes d'accès sur le clavier d'entrée, nous avons configuré un middleware de limitation de débit (Rate Limiting) sous Laravel, limitant les requêtes à 5 tentatives par minute par adresse IP cliente.
2. **Gestion de la concurrence lors des réservations** : Si deux membres cliquent simultanément sur le dernier ticket de cours libre, une condition de concurrence pouvait survenir. Nous avons résolu cela en utilisant des transactions SQL de type verrouillage exclusif (SELECT FOR UPDATE) au niveau du contrôleur Laravel `CoursController`.

CONCLUSION DU CHAPITRE

L'implémentation de FITZONE a permis de concrétiser les diagrammes UML dans une application web fonctionnelle et sécurisée. L'intégration de nos frameworks a offert d'excellents résultats de réactivité. Nous allons maintenant dresser le bilan de ce projet dans la conclusion générale de ce rapport.

Conclusion Générale

BILAN DU PROJET

Ce projet de fin d'études a permis de concevoir et de réaliser la plateforme web de gestion sportive **FITZONE**. L'application répond à l'ensemble des exigences fonctionnelles définies dans notre cahier des charges, notamment en offrant un système de pointage physique sans badge physique, basé uniquement sur un code personnel à 6 chiffres unique et rapide à valider en base de données.

L'architecture logicielle retenue (Laravel API et React SPA) a démontré sa pertinence en termes de performances d'affichage et de séparation claire des responsabilités de développement.

LIMITES ET AMÉLIORATIONS FUTURES

Malgré les bons résultats obtenus, l'application présente des limites qui ouvrent la voie à des évolutions futures :

- Intégration d'un module de paiement en ligne direct par carte bancaire via les API Stripe ou PayPal.
- Conception d'une application mobile native (React Native) pour permettre aux membres d'obtenir des notifications de rappel pour leurs cours réservés.
- Couplage du système de pointage à un tourniquet physique par le biais de cartes contrôleurs Arduino ou Raspberry Pi.

COMPÉTENCES DÉVELOPPÉES

Ce projet a été particulièrement formateur. Il m'a permis d'approfondir mes connaissances sur la modélisation objet UML, de maîtriser le développement d'API REST sécurisées, et de comprendre les enjeux de la synchronisation de données en temps réel entre un frontend SPA et un backend de base de données relationnelle.

Références Bibliographiques

1. **React Documentation** : Documentation officielle de référence pour la création de composants et la gestion des hooks. (<https://react.dev>)
2. **Laravel Framework Documentation** : Guide complet sur le routage API, Eloquent ORM et l'écriture de requêtes transactionnelles. (<https://laravel.com>)
3. **UML Distilled (3rd Edition)** : Martin Fowler - Guide pratique pour la modélisation des cas d'utilisation et des diagrammes de classes.
4. **MySQL Database Manual** : Référence pour l'optimisation des index et la gestion des verrous d'écriture concurrents. (<https://mysql.com>)

Annexes

Annexe A : Configuration du middleware de pointage sous Laravel

```
Route::middleware('throttle:5,1')->group(function () {  
    Route::post('/pointage/valider', [PointageController::class, 'valider']);  
});
```

Ce paramétrage protège l'API de pointage en bloquant temporairement les requêtes d'une adresse IP si plus de 5 tentatives de validation de code erronées sont émises en moins d'une minute.

